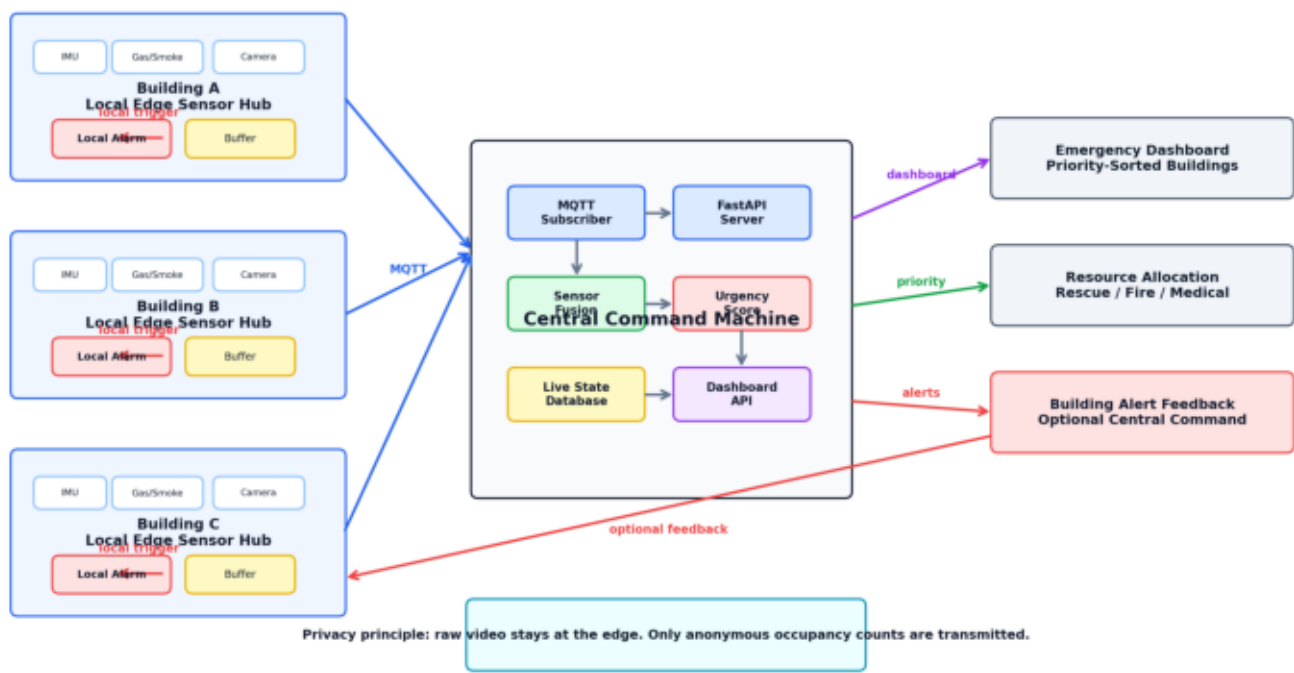


A-RES Hybrid Edge and Central Command System

Hybrid edge reaction + central emergency coordination

A-RES Hybrid Edge + Central Command Architecture

Local sensor hubs react immediately; central command coordinates all buildings.



System Concept

In the A-RES system, each building collects data from its own sensors, processes the data through a local edge hub, and sends it to a central command machine. The central system combines data from all buildings and calculates emergency response priorities. However, critical alarm decisions inside the building should not depend only on the central system. During an earthquake, internet disruption, network delay, or loss of connection to the central machine may occur.

For this reason, the most reliable architecture for A-RES is a **hybrid edge + central command** approach.

- Local edge hub = fast building-level reaction
- Central command machine = multi-building coordination and prioritization

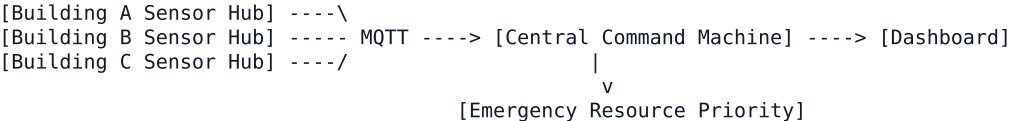
Why a Fully Centralized System Is Not Enough

In a fully centralized system, sensor data first travels from the building to the central machine. The central machine evaluates the data and then sends a command back to trigger the building alarm. This can work under normal conditions, but it creates risk during an earthquake because communication may be delayed or interrupted.

Therefore, the local sensor hub inside each building should be able to trigger alarms without waiting for the central command system in critical situations. For example, if very strong shaking, smoke, gas leakage, or sudden temperature increase is detected, the building's siren, warning lights, or local alert system can be triggered immediately.

The central system, on the other hand, evaluates all buildings together and determines where emergency teams should go first.

General Architecture



Each building contains the following components:

Component	Purpose
Accelerometer / IMU	Measures building shaking intensity
Smoke / gas sensors	Detect smoke, fire, or gas leakage risks
Temperature / environmental sensors	Support fire and environmental anomaly detection
Entrance camera counter	Estimates the number of people inside the building
Local edge hub	Collects sensor data, performs initial checks, and sends MQTT messages
Local alarm output	Triggers building alarms in critical conditions

Data Flow

The system works through two parallel decision flows.

1. Local Building-Level Reaction

This flow allows the building to react quickly by itself.

```
Accelerometer / smoke / gas sensor
  -> local edge hub
  -> immediate danger check
  -> local siren / light / warning trigger
```

Example:

```
If PGA > critical threshold OR gas_detected = true:
  trigger local alarm
```

This decision does not need to wait for a response from the central system.

2. Central Command Evaluation

This flow allows all buildings to be evaluated together.

Sensor data + occupancy data

```
-> MQTT
-> central FastAPI server
-> sensor fusion
-> urgency score
-> dashboard and emergency team priority
```

The central system answers questions such as:

- Which building experienced stronger shaking?
- Which building may still have more people inside?
- Which building has fire, smoke, or gas risk?
- Which building is more vulnerable?
- Where should rescue teams go first?

Occupancy Counting with Camera Processing

Cameras placed at building entrances are used to estimate the number of people inside. The system does not perform face recognition or identity detection. It only counts entry and exit movements.

Camera processing flow:

```
Entrance video
-> YOLO11n person detection
-> ByteTrack person tracking
-> virtual line crossing
-> entry / exit count
-> current occupancy
-> MQTT message to central machine
```

Example:

```
Person enters -> delta = +1
Person exits  -> delta = -1
```

This information is very important after an earthquake. If two buildings experienced similar shaking, the building with more people inside should receive higher priority.

Why We Use Laptop Processing Instead of Mobile On-Device Processing

At the beginning, we considered using the phone camera directly inside the mobile application to perform people counting. However, for the MVP, this approach creates more technical risk. Building a real-time camera, YOLO model, and tracking pipeline inside Flutter would take more time and could be unstable during the demo.

Therefore, we selected a more reliable MVP approach:

```
Phone records doorway video
-> laptop runs YOLO11n + ByteTrack
-> laptop publishes occupancy event via MQTT
-> central server receives data
-> dashboard updates building occupancy and urgency score
```

In this approach, the phone is still used as a camera or video source. The AI processing is performed on the laptop. In a real deployment, the laptop can be replaced by a Raspberry Pi, Jetson Nano, or mini PC as an edge device.

Relationship Between Sensors

A-RES does not make decisions based on a single sensor. Data from different sensors is evaluated together.

Sensor Combination	Interpretation
High PGA + high occupancy	Many people may be at risk after strong shaking
Medium PGA + gas detected	Gas leakage increases urgency
Low PGA + smoke detected	Fire risk may still require response
High PGA + old building	Vulnerable building receives higher priority
Stale data + previous high risk	System behaves cautiously when data is outdated

Based on this combined evaluation, the central system calculates an urgency score for each building.

Conclusion

This design is logical and creates a strong system architecture for A-RES. The best way to

describe it is:

Local sensor hubs detect immediate dangerous conditions and can trigger local building alarms, while the central command system collects data from all buildings, calculates urgency scores, and coordinates emergency response.

This structure provides both fast local building alerts and centralized disaster coordination.